GPU Depreciation Cycle

An illustrative scenario model of how fast data-centre GPUs lose value, and why the useful-life assumption matters as much as the decay rate.

Ajai Upadhyaya

ABSTRACT This report works through a scenario model of GPU value decay. Every input is an assumption, not observed market data. In the base scenarios an H100 loses 30.2% of its value in its first year and an A100 loses 38.1%, with half-lives of 23 and 17 months. Across the model's thirteen decay assumptions, implied first-year loss runs from 22% to 47%. Because exponential decay front-loads the loss, a six-year straight-line schedule would carry H100 book value up to 18 points above modelled value. The life that matches first-year loss is 3.3 years for the H100 and 2.6 for the A100.

01

Summary

How quickly does an AI accelerator lose its value, and what follows if the books assume it lasts longer? This report answers both questions inside a single scenario model. It does not measure the market. It makes the consequences of each assumption explicit, so they can be argued about one at a time.

30.2%	38.1%	22–47%	18 pts
First-year value loss, H100 base scenario (k = 0.030 / month)	First-year value loss, A100 base scenario (k = 0.040 / month)	Range of first-year loss implied by the model's thirteen decay assumptions	Peak gap, six-year straight-line book value over modelled H100 value

KEY FINDINGS

1. **The base case halves an H100 in about two years.** Exponential decay at 3.0% a month leaves 69.8% of new value after 12 months, 48.7% after 24 and 34.0% after 36, a half-life of 23.1 months. The older A100, at 4.0% a month, halves in 17.3 months.
2. **The assumption is the result.** The model contains thirteen separate decay assumptions: six scenario curves, four vintage cohorts, a value curve, a Monte Carlo drift and a uniform range. They imply first-year losses from 22% to 47%. Choosing among them moves the answer more than any other input does.
3. **Loss is front-loaded, schedules are not.** In the H100 base case the first year takes 30.2 points of value, the second 21.1 and the third 14.7. A straight-line schedule that matches the first year needs a life of 3.3 years. On a six-year life, book value exceeds modelled value by up to 18 points (month 26), or 28.5 points for the A100.
4. **Utilisation dominates the unit economics.** Renting at the model's mean hyperscaler rate of $4.96 an hour, a reseller needs $10.09 of revenue per GPU-hour at 50% utilisation and $5.05 at full utilisation. In the model's driver equation, utilisation also produces the widest swing in the depreciation rate, ±3.2 points.

The project began as an op-ed on the GPU depreciation cycle and its consequences for AI companies and their investors. The repository contains the model and its charts, but no data on prices, fleets or company accounts. The investor angle below (Section 05) is therefore an argument from the model's mechanics, labelled as such throughout, not evidence about any firm.

02

What the model is, and is not

The source is a single Python script, `gpu_econ_viz.py` (797 lines). It contains eighteen chart functions and one exporter, which writes the residual-value scenarios to a JSON file for an interactive D3 page. The script describes itself as a "real data model" and titles one chart "Real GPU Price Distribution (2026 Market)". Nothing in the repository supports those labels: there is no dataset, no source, and no citation. Rental prices are uniform random draws within chosen ranges, decay rates are chosen constants, and the driver relationships are equations with chosen coefficients.

Table 1. What the repository contains.

Item	Content
gpu_econ_viz.py	Seeds NumPy with 42, draws four rental-price samples, and defines 18 chart functions plus the D3 exporter. No data is read from disk.
d3_gpu_depreciation.json	222 records: two GPUs × three scenarios × 37 months, each value equal to $100 \cdot e^{-km}$ rounded to three decimals (checked for every record).
gpu_depreciation_d3.html	Interactive page that toggles base, bull and stress curves.
18 PNG exports	Screenshots of the script's charts, mostly 1991 × 620 Plotly strips. Not reused here; the figures below are redrawn from the script.
Prose, data, tests	None.

The results should be read conditionally: *if* GPU value decays at these rates, *then* the following follows. Read that way, the model still makes the shape of the problem visible: exponential loss, uncertainty that widens with age, a mismatch with straight-line accounting, and the assumptions that decide the answer.

03

Model and assumptions

The core object is a residual-value index that starts at 100 when a GPU is deployed and decays exponentially at a constant monthly rate k. Two quantities follow directly from k: the half-life, and the share of value lost in the first year.

$$V(m) = 100 \cdot e^{-k\,m} \qquad \textit{half-life} = \ln 2 \,/\, k \qquad \textit{first-year loss} = 1 - e^{-12k} \tag{1}$$

Table 2. Assumption register. Every value is hard-coded in gpu_econ_viz.py; none has a cited source.

Component	Assumption	Lines
Rental prices, $ / GPU-hour	Uniform draws, n = 1,000 each: H100 hyperscaler U(3, 7), mean 4.96; H100 neocloud U(1.3, 2.5), mean 1.91; A100 hyperscaler U(2.5, 4), mean 3.25; A100 neocloud U(0.8, 1.8), mean 1.29	29–32
Provider spread	AWS = H100 hyperscaler mean; GCP × 1.2; Azure × 1.4; Vast = neocloud mean; RunPod × 1.1. The multipliers are invented, not list prices.	198–221
Contract term structure	Discount multipliers for 1–24-month terms, e.g. AWS 1.00 → 0.84, neocloud 1.00 → 0.76	346–373
Scenario decay, per month	H100: bull 0.022, base 0.030, stress 0.041. A100: bull 0.031, base 0.040, stress 0.053	743–754
Fan chart	700 log-random-walk paths over 36 months; drift and volatility H100 0.030 / 0.030, A100 0.040 / 0.038	267–339
Vintage cohorts, per month	2023 fleet 0.047; 2024 0.039; 2025 0.032; 2026 0.028	530–557
Value curve and Monte Carlo	$100 \cdot e^{-0.3t}$ over 5 years; Monte Carlo drift −0.3, volatility 0.2 a year	169–260
12-month loss distribution	Annual drop drawn U(22%, 42%), independent of price or segment	564–598
Unit economics	Break-even: rental cost = H100 hyperscaler mean, 0.7 kW at $0.12/kWh. ROI: revenue 15 × utilisation $/hr. Throughput 2,000 tokens/s	90–140, 429–466
Driver equation	rate = 0.06 + 0.12(1 − util) + 0.08 energy + 0.10 supply − 0.07 demand + 0.09 competition + ε; util ~ Beta(3, 2), energy ~ U(0.06, 0.24), supply ~ N(1, 0.15), demand ~ N(1.2, 0.2), competition ~ U(0.7, 1.4)	380–422
Other constructions	Supply and demand growth curves, an architecture-shock event study, a volatility-regime timeline and a liquidity scatter, each generated from stated formulas plus noise	146–163, 473–499, 649–732

04

How fast value decays

Scenario curves

The three scenarios per GPU differ only in k. Bull scenarios decay slowest, stress scenarios fastest, and the A100 runs about one percentage point a month faster than the H100 in each case. That gap is the model's encoding of the older chip's weaker position.

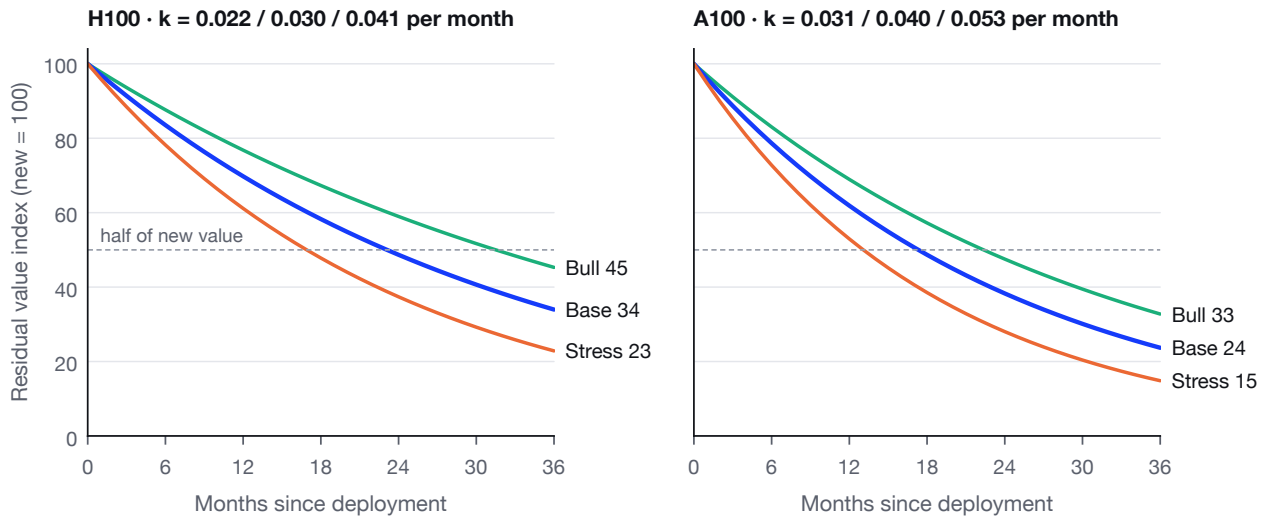


Figure 1. Residual-value scenarios, new = 100. The dashed line marks half of new value; every base and stress curve crosses it within 24 months.

Source: gpus-substack/d3_gpu_depreciation.json (222 records), redrawn.

Table 3. Scenario results. Losses are in points of new value (new = 100).

Scenario	k / month	Month 12	Month 24	Month 36	Half-life (mo)	Year 1 loss	Year 2 loss	Year 3 loss
H100 bull	0.022	76.8	59.0	45.3	31.5	23.2	17.8	13.7
H100 base	0.030	69.8	48.7	34.0	23.1	30.2	21.1	14.7
H100 stress	0.041	61.1	37.4	22.9	16.9	38.9	23.8	14.5
A100 bull	0.031	68.9	47.5	32.8	22.4	31.1	21.4	14.8
A100 base	0.040	61.9	38.3	23.7	17.3	38.1	23.6	14.6
A100 stress	0.053	52.9	28.0	14.8	13.1	47.1	24.9	13.2

Two regularities stand out. The first year always takes the largest bite, between 23 and 47 points. By the third year every scenario loses roughly the same 13–15 points, because fast-decaying scenarios have less value left to lose. And the spread between bull and stress cases is wide: an H100 is worth anywhere from 22.9 to 45.3 after three years, a factor of two, purely as a function of the chosen rate.

Every assumption in one place

The scenario curves are not the only decay assumptions in the script. The vintage cohorts, the five-year value curve, the Monte Carlo drift and the uniform 12-month distribution each encode a rate of their own. Put on a common scale, the share of value lost in the first twelve months, they run from 22% to 47%. The most pessimistic assumption more than doubles the first-year loss of the most optimistic.

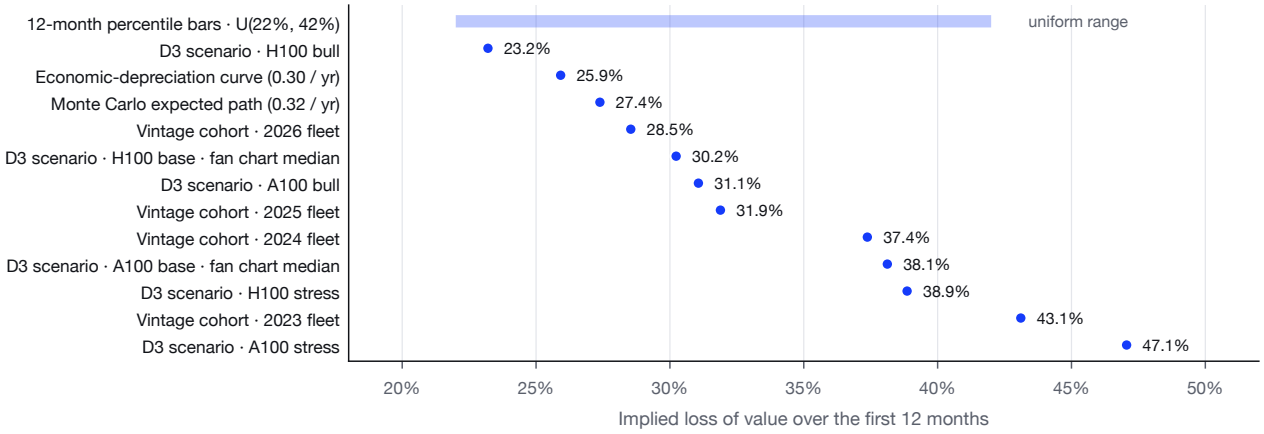


Figure 2. Implied first-year value loss for each decay assumption in the script, using $1 - e^{-12k}$ for monthly rates and $1 - e^{-r}$ for annual ones.

Source: gpu_econ_viz.py, lines 169–260, 267–339, 530–557, 564–598 and 743–754; derived for this report.

The newest vintage (2026, 0.028 a month) keeps 36.5% of its value after three years; the oldest (2023, 0.047) keeps 18.4%. The value curve and the Monte Carlo's expected path both end year five near a fifth of new value, at 22.3 and 20.2. Taken together, the script implicitly brackets first-year loss between about a fifth and a half of value. That bracket is a fair statement of the uncertainty the model expresses. A single headline number would hide it.

Uncertainty around a single scenario

The fan chart adds randomness to the base rates. Each of 700 paths takes monthly log-returns with drift $-k$ and a normal shock, so the median path equals the deterministic curve and the band widens with the square root of age.

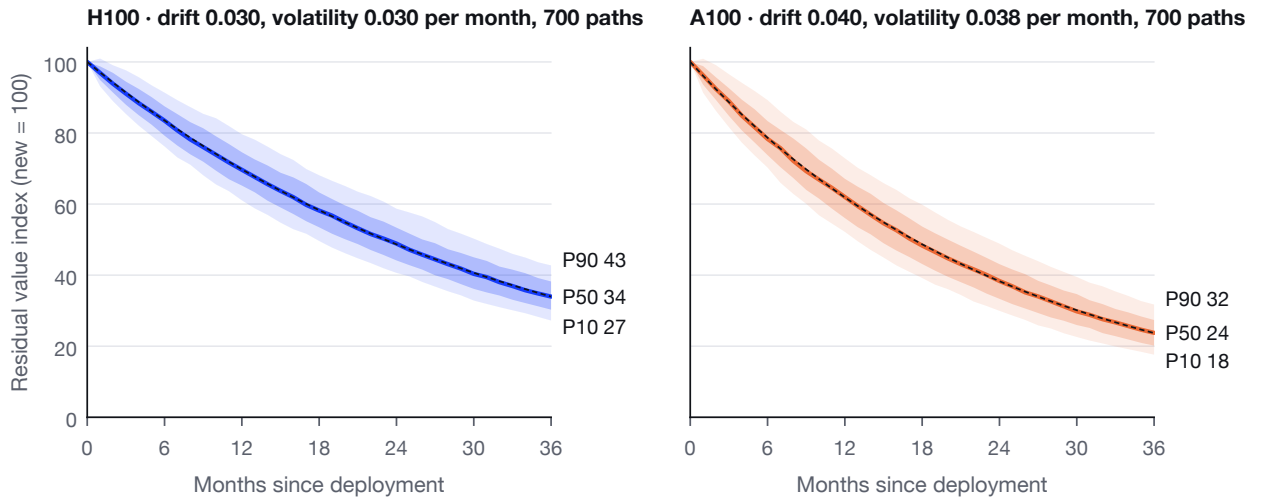


Figure 3. Simulated residual value, with 10th–90th (light) and 25th–75th (dark) percentile bands and the median. The dashed line is the deterministic base curve.

Source: depreciation_fan_chart() parameters (gpu_econ_viz.py, lines 267–339), re-simulated with seed 42 for this report.

After 36 months the H100's 10th to 90th percentile range is 27.2 to 42.8, against a median of 34.0. The A100's is 17.6 to 31.7 around 23.8. At month 36 these match the closed-form lognormal quantiles (26.96 and 42.77 for the H100) to within 0.3 points. The band spans about −20% to +26% of the median for the H100 and −26% to +34% for the A100. Both are narrower than the gap between the bull and stress scenarios.

05

Book value against modelled value

Owners of GPUs record them at cost and depreciate them over an assumed useful life, usually in equal annual amounts (straight-line). The model's value path is exponential. It loses most in the first year, while a straight-line schedule loses the same amount every year. This section compares the two, both expressed in index points with cost = 100.

> INFERENCE
>
> This section reasons from the model's mechanics. It does not use any company's accounts, and it does not claim that any company's useful-life assumption is right or wrong.

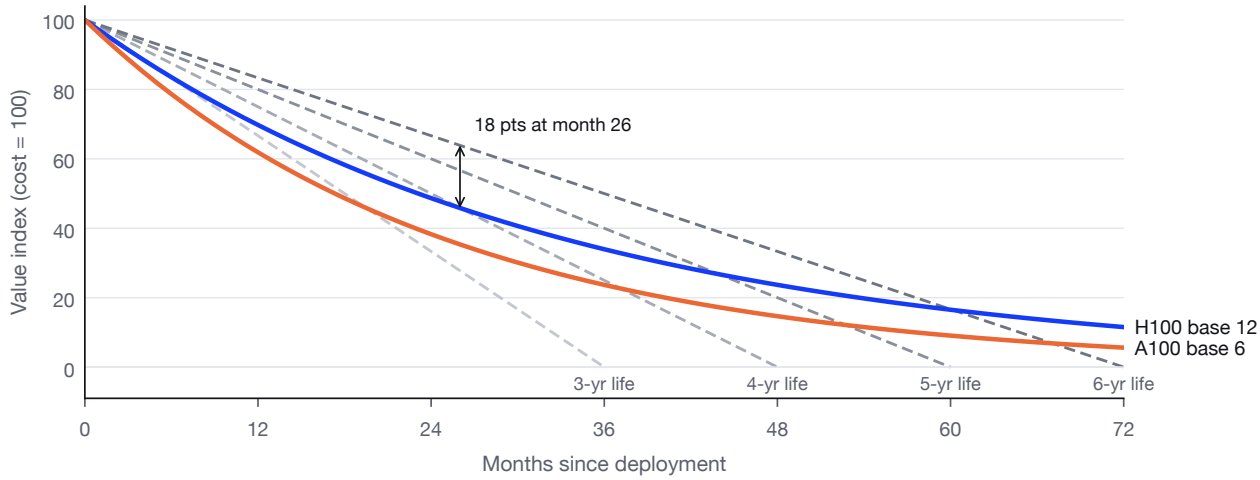


Figure 4. Straight-line book value for 3–6-year useful lives (dashed) against the model's base-case value paths. The arrow marks the largest gap between the six-year schedule and the H100 path.

Source: scenario rates from gpu_econ_viz.py, lines 743–754; straight-line schedules and gaps derived for this report.

Table 4. Book value minus modelled value at month 24, in index points, by useful life. Positive means book value is above modelled value.

Scenario	3 years	4 years	5 years	6 years	Peak gap, 6 years	Matching life
H100 bull	−25.6	−9.0	+1.0	+7.7	7.8 (m21)	4.3 yrs
H100 base	−15.3	+1.3	+11.3	+18.0	18.0 (m26)	3.3 yrs
H100 stress	−4.0	+12.6	+22.6	+29.3	29.5 (m26)	2.6 yrs
A100 bull	−14.2	+2.5	+12.5	+19.1	19.2 (m26)	3.2 yrs
A100 base	−5.0	+11.7	+21.7	+28.4	28.5 (m26)	2.6 yrs
A100 stress	+5.3	+22.0	+32.0	+38.6	38.7 (m25)	2.1 yrs

Matching life = the straight-line life whose first-year charge equals the scenario's first-year loss, $1 / (1 - e^{-12k})$.

The pattern is consistent. A straight-line life that matches the model's first-year loss is short: 3.3 years for the H100 base case, 2.6 for the A100 base case, and as little as 2.1 years under A100 stress. Only the H100 bull case supports a life above four years on this test. At five or six years, book value sits above modelled value through the middle of the asset's life. In the base cases the largest gap comes between month 20 and month 26. After that it narrows as the schedule catches up.

If the model's decay rates were right, three consequences would follow. These are inferences, not observations. First, early-year depreciation expense would understate the value actually being consumed, flattering reported margins while the fleet is young. Second, a resale or write-down in the middle years would realise a loss against book value, about 18 points of cost for an H100 sold at month 26 on a six-year life. Third, the useful-life choice would matter as much as the decay rate. Moving from a six-year to a four-year life changes the H100 base-case gap at month 24 from +18.0 to +1.3 points. Moving from the base to the stress decay rate on a four-year life changes it from +1.3 to +12.6.

06

Unit economics and sensitivity

Break-even revenue

The script's break-even frontier models a reseller that rents a GPU, pays for its power, and sells inference by the hour. Hourly profit is revenue × utilisation − rental cost − 0.7 kW × $0.12/kWh, so the break-even revenue per GPU-hour is (rental + $0.084) / utilisation. The script uses the hyperscaler mean rate; the neocloud line applies the same formula to the script's neocloud mean, as a variant.

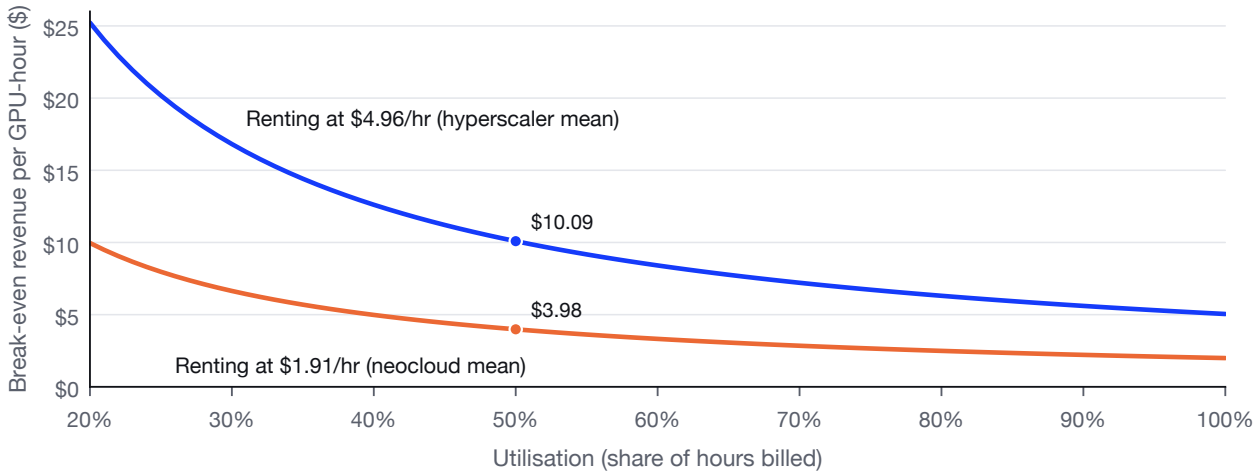


Figure 5. Break-even revenue per GPU-hour by utilisation, for an H100 rented at the script's mean hyperscaler ($4.96) and neocloud ($1.91) rates.

Source: `break_even_profit_contour()` inputs (`gpu_econ_viz.py`, lines 429–466); neocloud variant derived for this report.

At 30% utilisation the reseller needs $16.82 per GPU-hour on hyperscaler capacity, falling to $10.09 at 50%, $6.31 at 80% and $5.05 at full use. On neocloud capacity the same thresholds are $6.64, $3.98, $2.49 and $1.99. Power is a small term here: in the script's ROI surface, raising electricity from $0.05 to $0.25/kWh lowers the return multiple by only about 7%. Across the surface the multiple runs from 0.27 to 2.96, driven almost entirely by utilisation. On the script's throughput assumption of 2,000 tokens per second, compute costs $0.69 per million tokens at the hyperscaler mean and $0.27 at the neocloud mean.

What moves the depreciation rate

The script's driver equation sets an annual depreciation rate from five inputs. Moving each input from its 10th to its 90th percentile, with the others held at their means, shows how much each matters within the model. The mean rate is about 23%.

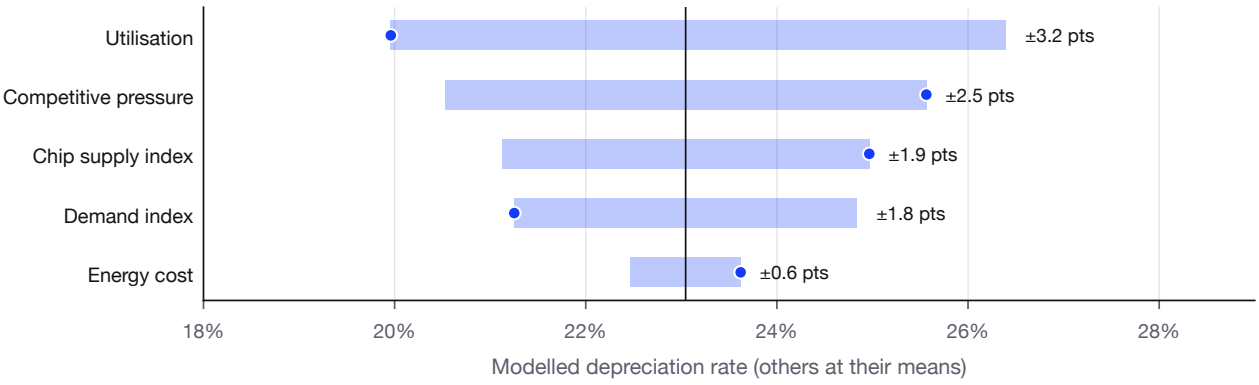


Figure 6. Range of the modelled depreciation rate as each driver moves from its 10th to 90th percentile. The dot marks the 90th-percentile end; the vertical line is the mean rate.

Source: coefficients and distributions of `depreciation_driver_correlation()` (`gpu_econ_viz.py`, lines 380–422); swings derived for this report.

Utilisation dominates (±3.2 points), followed by competitive pressure (±2.5), chip supply (±1.9) and demand (±1.8). Energy cost barely registers (±0.6). Higher utilisation and stronger demand slow depreciation; more supply and more competition speed it up. These magnitudes restate the script's assumed coefficients: they show what the model believes, not what the market does. The script's separate tornado chart uses bar lengths typed in directly, which do not follow from this equation and include a driver, model efficiency gain, that the equation lacks.

07

Limitations

- **No data.** Every price, rate and coefficient is an unsourced assumption; the script's "real" and "2026 market" labels are not supported.
- **Prices and value are not linked.** Rental prices are drawn independently of the decay rates, and with no purchase price, value is an index, not dollars.
- **Constant decay.** One rate per scenario rules out step changes at new-architecture launches; the script's event study sketches one with invented magnitudes.
- **Circular relationships.** Driver correlations, the liquidity trend and the identical 12-month distributions are built into the generating formulas.
- **Code issues in the source.** The Monte Carlo function draws an independent shock at every point instead of accumulating them, so its lines are not paths. The fan chart's legend names only the H100. Outputs also depend on the order in which functions run, because they share one random state, so the figures here are redrawn with their own seeds.

A calibrated version would need observed resale prices for GPUs by model and age, rental-rate histories by provider and term, and owners' disclosed useful lives. The decay rates in Table 2 could then be estimated rather than chosen, and Section 05 would become a test instead of an illustration.

A

Source and reproducibility

REPOSITORY	ajaiupadhyaya/gpus-substack, commit dea43eb, 17 May 2026.
MODEL	gpu_econ_viz.py (797 lines), seeded with numpy.random.seed(42) at import.
SCENARIO DATA	d3_gpu_depreciation.json, 222 records. Every value was checked against $100 \cdot e^{-km}$ to three decimals.
PRICES	Taken by importing the script, which reproduces its seeded draws; the H100 means ($4.96, $1.91) match the repository's exported chart.
THIS REPORT	Six figures redrawn from the script's parameters and the JSON file. Fan chart re-simulated (700 paths, seed 42) and checked against closed-form quantiles; driver swings from 2,000,000 draws per input (seed 7). Every other number is a closed-form result derived here. The repository's PNG exports were not reused.

Ajai Upadhyaya · esnupi Publications No. 02 · October 2026 · Set in Charter, Archivo Black and Courier New.